



2

THE WINDOWS KERNEL



Windows is a secure, multiuser operating system.

However, it's also one of the most challenging modern operating systems to understand in detail. Before we delve into the intricacies of its security, in this part of the book I'll provide you with an overview of the operating system's structure. We'll also take this opportunity to understand how to use the PowerShell module that will form the core

of this book.

We'll consider the two parts of the running operating system: the kernel and the user-mode applications. The kernel makes the security decisions that determine what a user can do on the system. However, most of the applications you use on a Windows machine run in user mode. This chapter will focus on the kernel; the next chapter will focus on user-mode applications.

In the following sections, we'll examine the various subsystems that make up the Windows kernel. For each subsystem, I'll explain its purpose and how it's used. We'll begin with the object manager, where we'll also explore system calls, which allow a user-mode application to access kernel objects. We'll then discuss the input/output manager, how applications are created through the process and thread manager, and how memory is represented with the memory manager. Throughout, I'll outline how you can inspect the behavior of these subsystems using PowerShell.

The Windows Kernel Executive

The *Windows NTOS kernel executive*, or *kernel* for short, is the heart of Windows. It provides all the operating system's privileged functionality, as well as interfaces through which the user applications can communicate with the hardware. The kernel is split into multiple subsystems, each with a dedicated purpose. **Figure 2-1** shows a diagram of the components in which we'll be most interested in this book.

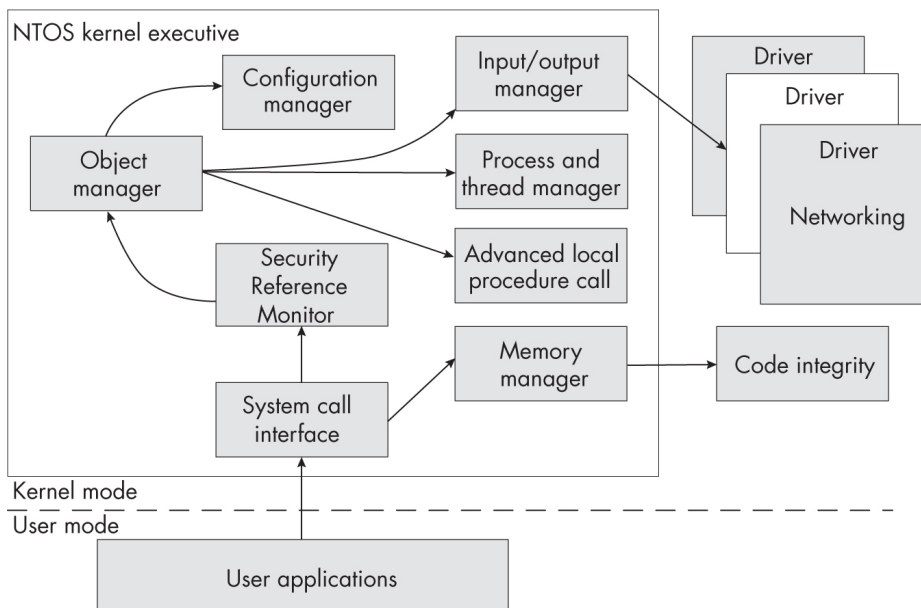


Figure 2-1: The Windows kernel executive modules

Each subsystem in the kernel executive exposes APIs for other subsystems to call. If you are looking at kernel code, you can quickly determine what subsystem each API belongs to using its two-character prefix. The prefixes for the subsystems in **Figure 2-1** are shown in **Table 2-1**.

Table 2-1: API Prefix-to-Subsystem Mapping

Prefix	Subsystem	Example
Nt or Zw	System call interface	NtOpenFile/ZwOpenFile
Se	Security Reference Monitor	SeAccessCheck
Ob	Object manager	ObReferenceObjectByHandle
Ps	Process and thread manager	PsGetCurrentProcess

Prefix	Subsystem	Example
Cm	Configuration manager	CmRegisterCallback
Mm	Memory manager	MmMapIoSpace
Io	Input/output manager	IoCreateFile
Ci	Code integrity	CiValidateFileObject

We'll explore all of these subsystems in the sections that follow.

The Security Reference Monitor

For the purposes of this book, the *Security Reference Monitor (SRM)* is the most important subsystem in the kernel. It implements the security mechanisms that restrict which users can access different resources. Without the SRM, you wouldn't be able to prevent other users from accessing your files. [Figure 2-2](#) shows the SRM and its related system components.

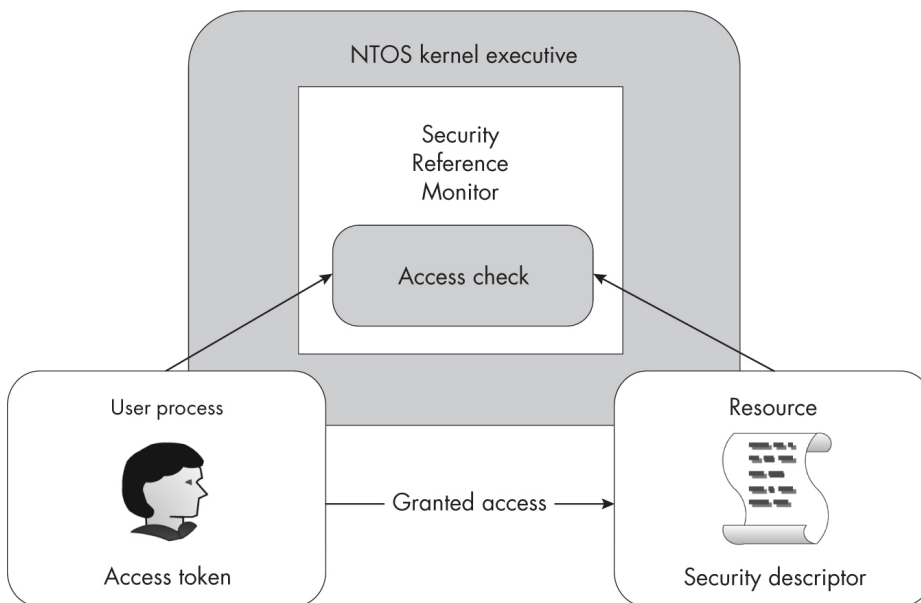


Figure 2-2: Components of the Security Reference Monitor

Every process running on the system is assigned an *access token* when it's created. This access token is managed by the SRM and defines the identity of the user associated with that process. The SRM can then perform an operation called an *access check*. This operation queries a resource's security descriptor, compares it to the process's access token, and either calcu-

lates the level of granted access or indicates that access is denied to the caller.

The SRM is also responsible for generating audit events whenever a user accesses a resource. Auditing is disabled by default due to the volume of events it can produce, so an administrator must enable it first. These audit events can be used to identify malicious behavior on a system as well as to diagnose security misconfigurations.

The SRM expects users and groups to be represented as binary structures called *security identifiers (SIDs)*. However, passing around raw binary SIDs isn't very convenient for users, who normally refer to users and groups by meaningful names (for example, the user *bob* or the *Users* group). These names need to be converted to SIDs before the SRM can use them. The task of name-SID conversion is handled by the *Local Security Authority Subsystem (LSASS)*, which runs inside a privileged process independent from any logged-in users.

It's infeasible to represent every possible SID as a name, so Microsoft defines the *Security Descriptor Definition Language (SDDL)* format to represent a SID as a string. SDDL can represent the entire security descriptor of a resource, but for now we'll just use it to represent the SID. In [Listing 2-1](#), we use PowerShell to look up the *Users* group name using the `Get-NtSid` command; this should retrieve the SDDL string for the SID.

```
PS> Get-NtSid -Name "Users"
Name          Sid
----          ---
BUILTIN\Users S-1-5-32-545
```

Listing 2-1: Querying the SID of the Users group using Get-NtSid

We pass the name of the *Users* group to `Get-NtSid`, which returns the fully qualified name, with the local domain *BUILTIN* attached. The *BUILTIN\Users* SID is always the same between different Windows systems. The output also contains the SID in SDDL format, which can be broken down into the following dash-separated parts:

- The *s* character prefix. This indicates that what follows is an SDDL SID.
- The version of the SID structure in decimal. This has a fixed value of 1.
- The security authority. Authority 5 indicates the built-in NT authority.
- Two relative identifiers (RIDs), in decimal. The RIDs (here, 32 and 545) represent the NT authority group.

We can also use `Get-NtSid` to perform the reverse operation, converting an SDDL SID back to a name, as shown in [Listing 2-2](#).

```
PS> Get-NtSid -Sddl "S-1-5-32-545"
Name          Sid
----          -
BUILTIN\Users S-1-5-32-545
```

Listing 2-2: Using `Get-NtSid` to find the name associated with a SID

I'll describe the SRM and its functions in much greater depth in [Chapters 4](#) through [9](#), and we'll revisit the SID structure in [Chapter 5](#), when we discuss security descriptors. For now, remember that SIDs represent users and groups and that we can represent them as strings in SDDL form. Next, we'll move on to another of the core Windows kernel executive subsystems, the object manager.

The Object Manager

On Unix-like operating systems, everything is a file. On Windows, everything is an object, meaning that every file, process, and thread is represented in kernel memory as an object structure. Importantly for security, each of these objects can have an assigned security descriptor, which restricts which users can access the object and determines the type of access they have (for example, read or write).

The *object manager* is the component of the kernel responsible for managing these resource objects, their memory allocations, and their lifetimes. In this section, we'll first discuss the types of objects the object manager supports. Then, we'll explore how kernel objects can be opened through a naming convention using a system call. Finally, we'll look at how to use a handle returned by the system call to access the object.

Object Types

The kernel maintains a list of all the types of objects it supports. This is necessary, as each object type has different supported operations and security properties. [Listing 2-3](#) shows how to use the `Get-NtType` command to list all supported types in the kernel.

```
PS> Get-NtType
Name
----
```

```
Type
Directory
SymbolicLink
Token
Job
Process
Thread
--snip--
```

Listing 2-3: Executing Get-NtType

I've truncated the list of types (the machine I'm using supports 72 of them), but there are some noteworthy entries even in this short section. The first entry in the generated list is `Type`; even the list of kernel types is built from objects! Other types of note here are `Process` and `Thread`, which represent the kernel objects for a process and a thread, respectively. We'll examine other object types in more detail later in this chapter.

You can display the properties of a type with `Format-List`, which returns additional information about that type. We'll look at an example later, but for now the question is how to access each of these types. To answer it, we'll need to talk about the object manager namespace.

The Object Manager Namespace

As a user of Windows, you typically see your filesystem drives in Explorer. But underneath the user interface is a whole additional filesystem just for kernel objects. Access to this filesystem, referred to as the *object manager namespace (OMNS)*, isn't very well documented or exposed to most developers, which makes it even more interesting.

The OMNS is built out of `Directory` objects. The objects act as if they were in a filesystem, so each directory contains other objects, which you can consider to be files. However, they are distinct from the file directories you're used to. Each directory is configured with a security descriptor that determines which users can list its contents and which users can create new subdirectories and objects inside it. You can specify the full path to an object with a backslash-separated string.

We can enumerate the OMNS by using a drive provider that is part of this book's PowerShell module. As shown in [Listing 2-4](#), this exposes the OMNS as if it were a filesystem by listing the *NtObject* drive.

```
PS> ls NtObject:\ | Sort-Object Name
Name                TypeName
-----
ArcName             Directory
BaseNamedObjects   Directory
BindFltPort        FilterConnectionPort
Callback           Directory
CLDMSGPORT         FilterConnectionPort
clfs               Device
CsrSbSyncEvent     Event
Device             Directory
Dfs                SymbolicLink
DosDevices         SymbolicLink
--snip--
```

Listing 2-4: Listing the root OMNS directory

Listing 2-4 shows a short snippet of the root OMNS directory. By default, this output includes the name of each object and its type. We can see a few `Directory` objects; you can list them if you have permission to do so. We can also see another important type, `SymbolicLink`. You can use symbolic links to redirect one OMNS path to another. A `SymbolicLink` object contains a `SymbolicLinkTarget` property, which itself contains the target that the link should open. For example, **Listing 2-5** shows the target for a symbolic link in the root of the OMNS.

```
PS> ls NtObject:\Dfs | Select-Object SymbolicLinkTarget
SymbolicLinkTarget
-----
\Device\DfsClient

PS> Get-Item NtObject:\Device\DfsClient | Format-Table
Name                TypeName
-----
DfsClient          Device
```

Listing 2-5: Showing the target of a symbolic link

Here, we list the `|Dfs` OMNS path, then extract the `SymbolicLinkTarget` property to get the real target. Next, we check the target path, `Device\DfsClient`, to show it's a `Device` type, which is what the symbolic link can be used to access.

Windows preconfigures several important object directories, shown in [Table 2-2](#).

Table 2-2: Well-Known Object Directories and Descriptions

Path	Description
<i>\BaseNamedObjects</i>	Global directory for user objects
<i>\Device</i>	Directory containing devices such as mounted filesystems
<i>\GLOBAL??</i>	Global directory for symbolic links, including drive mappings
<i>\KnownDlls</i>	Directory containing special, known DLL mappings
<i>\ObjectTypes</i>	Directory containing named object types
<i>\Sessions</i>	Directory for separate console sessions
<i>\Windows</i>	Directory for objects related to the Window Manager
<i>\RPC Control</i>	Directory for remote procedure call endpoints

The first directory in [Table 2-2](#), *BaseNamedObjects* (BNO), is important in the context of the object manager. It allows any user to create named kernel objects. This single directory allows the sharing of resources between different users on the local system. Note that you don't have to create objects in the BNO directory; it's only a convention.

I'll describe the other object directories in more detail later in this chapter. For now, you can list them in PowerShell by prefixing the path with *NtObject:*, as I've shown in [Listing 2-5](#).

System Calls

How can we access the named objects in the OMNS from a user-mode application? If we're in a user-mode application, we need the kernel to access the objects, and we can call kernel-mode code in a user-mode application using the system call interface. Most system calls perform some op-

eration on a specific type of kernel object exposed by the object manager. For example, the `NtCreateMutant` system call creates a `Mutant` object, a mutual exclusion primitive used for locking and thread synchronization.

The name of a system call follows a common pattern. It starts with either `Nt` or `Zw`. For user-mode callers, the two prefixes are equivalent; however, if the system call is invoked by code executing in the kernel, the `Zw` prefix changes the security checking process. We'll come back to the implications of the `Zw` prefix in [Chapter 7](#), when we talk about access modes.

After the prefix comes the operation's verb: `Create`, in the case of `NtCreateMutant`. The rest of the name relates to the kernel object type the system call operates on. Common system-call verbs that perform an operation on a kernel object include:

create Creates a new object. Maps to `New-Nt<Type>` PowerShell commands.

open Opens an existing object. Maps to `Get-Nt<Type>` PowerShell commands.

QueryInformation Queries object information and properties.

SetInformation Sets object information and properties.

Certain system calls perform type-specific operations. For example, `NtQueryDirectoryFile` is used to query the entries in a `File` object directory. Let's look at the C-language prototype for the `NtCreateMutant` system call to understand what parameters need to be passed to a typical call. As shown in [Listing 2-6](#), the `NtCreateMutant` system call creates a new `Mutant` object.

```
NTSTATUS NtCreateMutant(  
    HANDLE* FileHandle,  
    ACCESS_MASK DesiredAccess,  
    OBJECT_ATTRIBUTES* ObjectAttributes,  
    BOOLEAN InitialOwner  
);
```

Listing 2-6: The C prototype for `NtCreateMutant`

The first parameter for the system call is an outbound pointer to a `HANDLE`. Common in many system calls, this parameter is used to retrieve an opened handle to the object (in this case, a `Mutant`) when the function succeeds. We use handles along with other system calls to access properties

and perform operations. In the case of our `Mutant` object, the handle allows us to acquire and release the lock to synchronize threads.

Next is `DesiredAccess`, which represents the operations the caller wants to be able to perform on the `Mutant` using the handle. For example, we could request access that allows us to wait for the `Mutant` to be unlocked. If we didn't request that access, any application that tried to wait on the `Mutant` would immediately fail. The access granted depends on the results of the SRM's access check. We'll discuss handles and `DesiredAccess` in more detail in the next section.

Third is the `ObjectAttributes` parameter, which defines the attributes for the object to open or create. The `OBJECT_ATTRIBUTES` structure is defined as shown in [Listing 2-7](#).

```
struct OBJECT_ATTRIBUTES {
    ULONG          Length;
    HANDLE          RootDirectory;
    UNICODE_STRING* ObjectName;
    ULONG           Attributes;
    PVOID           SecurityDescriptor;
    PVOID           SecurityQualityOfService;
}
```

Listing 2-7: The `OBJECT_ATTRIBUTES` structure

This C-language structure starts with `Length`, which represents the length of the structure. Specifying the structure length at the start is a common C-style idiom to ensure that the correct structure has been passed to the system call.

Next come `RootDirectory` and `ObjectName`. These are taken together, as they indicate how the system call should look up the resource being accessed. The `RootDirectory` is a handle to an opened kernel object to use as the base for looking up the object. The `ObjectName` field is a pointer to a `UNICODE_STRING` structure. This is a counted string, defined in [Listing 2-8](#) as a C-language structure.

```
struct UNICODE_STRING {
    USHORT Length;
    USHORT MaximumLength;
    WCHAR* Buffer;
};
```

Listing 2-8: The UNICODE_STRING structure

The structure references the string data through `Buffer`, which is a pointer to an array of 16-bit Unicode characters. The string is represented in UCS-2 encoding; Windows predates many of the changes to Unicode, such as UTF-8 and UTF-16.

The `UNICODE_STRING` structure also contains two length fields, `Length` and `MaximumLength`. The first length field represents the total valid length of the string pointed to by `Buffer`, in bytes (not in Unicode characters). If you're coming from a C programming background, this length does not include any NUL terminating character. In fact, a NUL character is permitted in object names.

The second length field represents the maximum length of the string pointed to by `Buffer`, in bytes. Because the structure has two separate lengths, it's possible to allocate an empty string with a large maximum length and a valid length of zero, then update the string value using the `Buffer` pointer. Note that the lengths are stored as `USHORT` values, which are unsigned 16-bit integers. Coupled with the length-representing bytes, this means a string can be at most 32,767 characters long.

To specify the name of an object, you have two options: you can set `ObjectName` to an absolute path of, for example, `|BaseNamedObjects|ABC`, or you can set `RootDirectory` to a `Directory` object for `|BaseNamedObjects` and then pass `ABC` as the `ObjectName`. These two actions will open the same object.

Returning to [Listing 2-7](#), after the `ObjectName` parameter comes `Attributes`, which is a set of flags to modify the object name lookup process or change the returned handle's properties. [Table 2-3](#) shows the valid values for the `Attributes` field.

Table 2-3: Object Attribute Flags and Descriptions

PowerShell name	Description
Inherit	Marks the handle as inheritable.
Permanent	Marks the handle as permanent.
Exclusive	Marks the handle as exclusive if creating a new object. Only the same process can

PowerShell name	Description
	open a handle to the object.
CaseInsensitive	Looks up the object name in a case-insensitive manner.
OpenIf	If using a <code>CreateIf</code> call, opens a handle to an existing object if available.
OpenLink	Opens the object if it's a link to another object; otherwise, follows the link. This is used only by the configuration manager.
KernelHandle	Opens the handle as a kernel handle when used in kernel mode. This prevents user-mode applications from accessing the handle directly.
ForceAccessCheck	When used in kernel mode, ensures all access checks are performed, even if calling the <code>zw</code> version of the system call.
IgnoreImpersonatedDeviceMap	Disables the device map when impersonating.
DontReparse	Indicates not to follow any path that contains a symbolic link.

The final two fields in the `OBJECT_ATTRIBUTES` structure allow the caller to specify the Security Quality of Service (SQoS) and security descriptor for the object. We'll come back to SQoS in [Chapter 4](#) and the security descriptor in [Chapter 5](#).

Next in the `NtCreateMutant` system call in [Listing 2-6](#) is the `InitialOwner` Boolean parameter, which is specific to this type. In this case, it represents whether the created `Mutant` is owned by the caller or not. Many other system calls, especially for files, have more complex parameters, which we'll discuss in more detail later in the book.

NTSTATUS Codes

All system calls return a 32-bit NTSTATUS code. This status code is composed of multiple components packed into the 32 bits, as shown in [Figure 2-3](#).

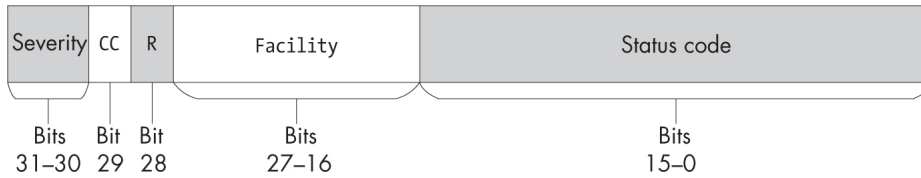


Figure 2-3: The NT status code structure

The most significant two bits (31 and 30) indicate the *severity* of the status code. [Table 2-4](#) shows the available values.

Table 2-4: NT Status Severity Codes

Severity name	Value
STATUS_SEVERITY_SUCCESS	0
STATUS_SEVERITY_INFORMATIONAL	1
STATUS_SEVERITY_WARNING	2
STATUS_SEVERITY_ERROR	3

If the severity level indicates a warning or error, then bit 31 of the status code will be set to 1. If the status code is treated as a signed 32-bit integer, this bit represents a negative value. It's a common coding practice to assume that if the status code is negative it represents an error, and if it's positive it represents a success. As we can see from the table, this assumption isn't completely true—the negative status code could also be a warning—but it works well enough in practice.

The next component in [Figure 2-3](#), CC, is the customer code. This is a single-bit flag that indicates whether the status code is defined by Microsoft (a value of 0) or defined by a third party (a value of 1). Third parties are not obliged to follow this specification, so don't treat it as fact.

Following the customer code is the R bit, a reserved bit that must be set to 0.

The next 12 bits indicate the *facility*—that is, the component or subsystem associated with the status code. Microsoft has predefined around 50 facilities for its own purposes. Third parties should define their own facility and combine it with the customer code to distinguish themselves from Microsoft. [Table 2-5](#) shows a few commonly encountered facilities.

Table 2-5: Common Status Facility Values

Facility name	Value	Description
FACILITY_DEFAULT	0	The default used for common status codes
FACILITY_DEBUGGER	1	Used for codes associated with the debugger
FACILITY_NTWIN32	7	Used for codes that originated from the Win32 APIs

The final component, the *status code*, is a 16-bit number chosen to be unique for the facility. It's up to the implementer to define what each number means. The PowerShell module contains a list of known status codes, which we can query using the `Get-NtStatus` command with no parameters ([Listing 2-9](#)).

```
PS> Get-NtStatus
Status      StatusName      Message
-----
00000000    STATUS_SUCCESS    STATUS_SUCCESS
00000001    STATUS_WAIT_1     STATUS_WAIT_1
00000080    STATUS_ABANDONED_WAIT_0 STATUS_ABANDONED_WAIT_0
000000C0    STATUS_USER_APC   STATUS_USER_APC
000000FF    STATUS_ALREADY_COMPLETE The requested action was completed by...
00000100    STATUS_KERNEL_APC STATUS_KERNEL_APC
00000101    STATUS_ALERTED    STATUS_ALERTED
00000102    STATUS_TIMEOUT    STATUS_TIMEOUT
00000103    STATUS_PENDING    The operation that was requested is p...
--snip--
```

Listing 2-9: Example output from Get-NtStatus

Notice how some status values, such as `STATUS_PENDING`, have a human-readable message. This message isn't embedded in the PowerShell module; in-

stead, it's stored inside a Windows library and can be extracted at runtime.

When we call a system call via a PowerShell command, its status code is surfaced through a .NET exception. For example, if we try to open a `Directory` object that doesn't exist, we'll see the exception shown in [Listing 2-10](#) displayed in the console.

```
PS> Get-NtDirectory \THISDOESNOTEXIST
❶ Get-NtDirectory : (0xC0000034) - Object Name not found.
--snip--

PS> Get-NtStatus 0xC0000034 | Format-List
Status           : 3221225524
❷ StatusSigned   : -1073741772
StatusName       : STATUS_OBJECT_NAME_NOT_FOUND
Message          : Object Name not found.
Win32Error       : ERROR_FILE_NOT_FOUND
Win32ErrorCode   : 2
Code             : 52
CustomerCode     : False
Reserved         : False
Facility         : FACILITY_DEFAULT
Severity         : STATUS_SEVERITY_ERROR
```

Listing 2-10: An NTSTATUS exception generated when trying to open a nonexistent directory

In [Listing 2-10](#), we use `Get-NtDirectory` to open the nonexistent path `THISDOESNOTEXIST`. This generates the `NTSTATUS 0xC0000034` exception, shown here along with the decoded message ❶. If you want more information about the status code, you can pass it to `Get-NtStatus` and format the output as a list to view all its properties, including `Facility` and `Severity`. The NT status code is an unsigned integer value; however, it's common to also see it printed (incorrectly) as a signed value ❷.

Object Handles

The object manager deals with pointers to kernel memory. A user-mode application cannot directly read or write to kernel memory, so how can it access an object? It does this using the handle returned by a system call, as discussed in the previous section. Each running process has an associated *handle table* containing three pieces of information:

- The handle's numeric identifier
- The granted access to the handle; for example, read or write
- The pointer to the object structure in kernel memory

Before the kernel can use a handle, the system call implementation must look up the kernel object pointer from the handle table using a kernel API such as `ObReferenceObjectByHandle`. By providing this handle indirectly, a kernel component can return the handle number to the user-mode application without exposing the kernel object directly. **Figure 2-4** shows the handle lookup process.

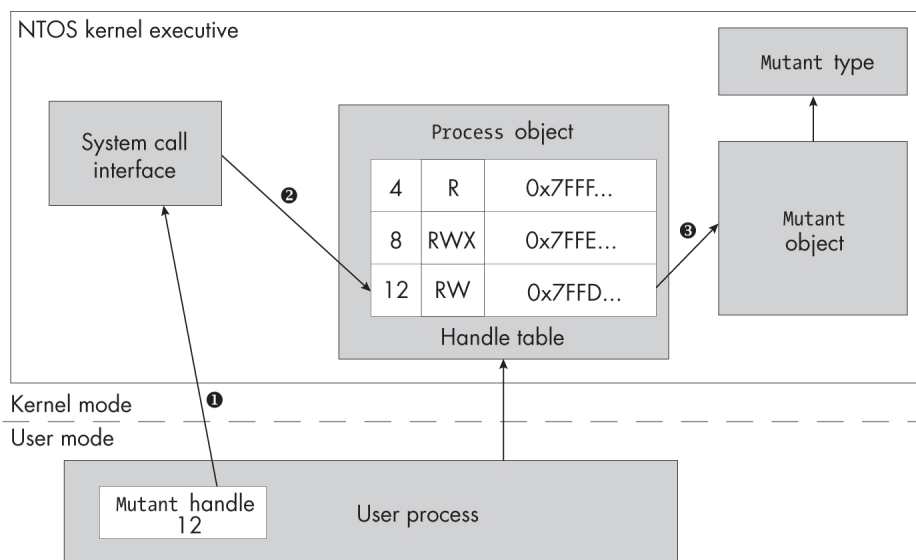


Figure 2-4: The handle table lookup process

In **Figure 2-4**, the user process is trying to perform some operation on a **Mutant object**. When a user process wants to use a handle, it must first pass the handle's value to the system call we defined in the previous section ①. The system call implementation then calls a kernel API to convert the handle to a kernel pointer by referencing the handle's numeric value in the process's handle table ②.

To determine whether to grant the access, the conversion API considers the type of access that the user has requested for the system call's operation, as well as the type of object being accessed. If the requested access doesn't match the granted access recorded in the handle table entry, the API will return `STATUS_ACCESS_DENIED` and the conversion operation will fail. Likewise, if the object types don't match ③, the API will return `STATUS_OBJECT_TYPE_MISMATCH`.

These two checks are crucial for security. The access check ensures that the user can't perform an operation on a handle to which they don't have

access (for example, writing to a file for which they have only read access). The type check ensures the user hasn't passed an unrelated kernel object type, which might result in type confusion in the kernel, causing security issues such as memory corruption. If the conversion succeeds, the system call now has a kernel pointer to the object, which it can use to perform the user's requested operation.

Access Masks

The granted access value in the handle table is a 32-bit bitfield called an *access mask*. This is the same bitfield used for the `DesiredAccess` parameter specified in the system call. We'll discuss how `DesiredAccess` and the access check process determine the granted access in more detail in [Chapter 7](#).

An access mask has four components, as shown in [Figure 2-5](#).

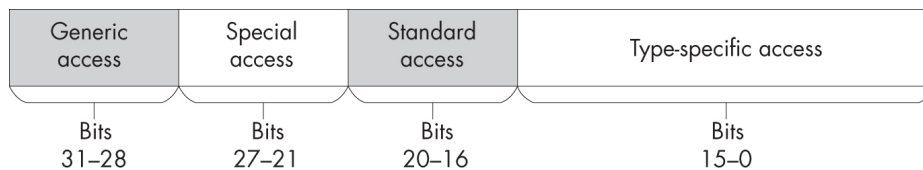


Figure 2-5: The access mask structure

The most important one is the 16-bit *type-specific access component*, which defines the operations that are allowed on a particular kernel object type. For example, a `File` object might have separate bits to specify whether the file is allowed to be read or written to when using the handle. In contrast, a synchronization `Event` might only have a single bit that allows the event to be signaled.

Working backward, the *standard access* component of the access mask defines operations that can apply to any object type. These operations include:

Delete Removes the object; for example, by deleting it from disk or from the registry

ReadControl Reads the security descriptor information for the object

WriteDac Writes the security descriptor's discretionary access control (DAC) to the object

WriteOwner Writes the owner information to the object

Synchronize Waits on the object; for example, waits for a process to exit or a mutant to be unlocked

We'll cover security-related access in more detail in [Chapters 5](#) and [6](#).

Before this are the *reserved* and *special access* bits. Most of these bits are reserved, but they include two access values:

AccessSystemSecurity Reads or writes audit information on the object

MaximumAllowed Requests the maximum access to an object when performing an access check

We'll cover **AccessSystemSecurity** access in [Chapter 9](#) and **MaximumAllowed** access in [Chapter 7](#).

Finally, the four high-order bits of the access mask (the *generic access* component) are used only when requesting access to a kernel object using the system call's **DesiredAccess** parameter. There are four broad categories of access: **GenericRead**, **GenericWrite**, **GenericExecute**, and **GenericAll**.

When you request one of these generic access rights, the SRM will first convert the access into the corresponding type-specific access. This means you'll never receive access to a handle with **GenericRead**; instead, you'll be granted access to the specific access mask that represents read operations for that type. To facilitate the conversion, each type contains a *generic mapping table*, which maps the four generic categories to type-specific access. We can display the mapping table using **Get-NtType**, as shown in [Listing 2-11](#).

```
PS> Get-NtType | Select-Object Name, GenericMapping
Name                                     GenericMapping
----
Type                                     R:00020000 W:00020000 E:00020000 A:000F0001
Directory                               R:00020003 W:0002000C E:00020003 A:000F000F
SymbolicLink                           R:00020001 W:00020000 E:00020001 A:000F0001
Token                                   R:0002001A W:000201E0 E:00020005 A:000F01FF
--snip--
```

Listing 2-11: Displaying the generic mapping table for object types

The type data doesn't provide names for each specific access mask. However, for all common types, the PowerShell module provides an enumerated type that represents the type-specific access. We can access this

type through the `Get-NtTypeAccess` command. [Listing 2-12](#) shows an example for the `File` type.

```
PS> Get-NtTypeAccess -Type File
```

Mask	Value	GenericAccess
----	-----	-----
00000001	ReadData	Read, All
00000002	WriteData	Write, All
00000004	AppendData	Write, All
00000008	ReadEa	Read, All
00000010	WriteEa	Write, All
00000020	Execute	Execute, All
00000040	DeleteChild	All
00000080	ReadAttributes	Read, Execute, All
00000100	WriteAttributes	Write, All
00010000	Delete	All
00020000	ReadControl	Read, Write, Execute, All
00040000	WriteDac	All
00080000	WriteOwner	All
00100000	Synchronize	Read, Write, Execute, All

Listing 2-12: Displaying the access mask for the File object type

The output of the `Get-NtTypeAccess` command shows the access mask value, the name of the access as known to the PowerShell module, and the generic access from which it will be mapped. Note how some access types are granted only to `All`; this means that even if you requested generic read, write, and execute access, you wouldn't be granted access to those rights.

SOFTWARE DEVELOPMENT KIT NAMES

To improve usability, the PowerShell module has modified the original names of the access rights found in the Windows software development kit (SDK). You can view the equivalent SDK names using the `SDKName` property with the `Get-NtTypeAccess` command:

```
PS> Get-NtTypeAccess -Type File | Select SDKName, Value
```

SDKName	Value
-----	-----
FILE_READ_DATA	ReadData
FILE_WRITE_DATA	WriteData
FILE_APPEND_DATA	AppendData

--snip--

These name mappings are useful for porting native code to PowerShell.

You can convert between a numeric access mask and specific object types using the `Get-NtAccessMask` command, as shown in [Listing 2-13](#).

```
PS> Get-NtAccessMask -FileAccess ReadData, ReadAttributes, ReadControl
Access
-----
00020081

PS> Get-NtAccessMask -FileAccess GenericRead
Access
-----
80000000

PS> Get-NtAccessMask -FileAccess GenericRead -MapGenericRights
Access
-----
00120089

PS> Get-NtAccessMask 0x120089 -AsTypeAccess File
ReadData, ReadEa, ReadAttributes, ReadControl, Synchronize
```

Listing 2-13: Converting access masks using `Get-NtAccessMask`

In [Listing 2-13](#), we first request the access mask from a set of `File` access names and receive the numeric access mask in hexadecimal. Next, we get the access mask for the `GenericRead` access; as you can see, the value returned is just the numeric value of `GenericRead`. We then request the access mask for `GenericRead` but specify that we want to map the generic access to a specific access by using the `MapGenericRights` parameter. As we've specified the access for the `File` type, this command uses the `File` type's generic mapping to convert to the specific access mask. Finally, we convert the raw access mask back to a type access using the `AsTypeAccess` parameter, specifying the kernel type to use.

As shown in [Listing 2-14](#), you can query an object handle's granted access mask through the PowerShell object's `GrantedAccess` property. This returns the enumerated type format for the access mask. To retrieve the numeric value, use the `GrantedAccessMask` property.

```
PS> $mut = New-NtMutant
PS> $mut.GrantedAccess
```

```
ModifyState, Delete, ReadControl, WriteDac, WriteOwner, Synchronize
```

```
PS> $mut.GrantedAccessMask
Access
-----
001F0001
```

Listing 2-14: Displaying the numeric value of the access mask using `GrantedAccessMask`

The kernel provides a facility to dump all handle table entries on the system through the `NtQuerySystemInformation` system call. We can access the handle table from PowerShell using the `Get-NtHandle` command, as illustrated in [Listing 2-15](#).

```
PS> Get-NtHandle -ProcessId $pid
```

ProcessId	Handle	ObjectType	Object	GrantedAccess
22460	4	Process	FFFF800224F02080	001FFFFF
22460	8	Thread	FFFF800224F1A140	001FFFFF
22460	12	SymbolicLink	FFFF9184AC639FC0	000F0001
22460	16	Mutant	FFFF800224F26510	001F0001
--snip--				

Listing 2-15: Displaying the handle table for the current process using `Get-NtHandle`

Each handle entry contains the type of the object, the address of the kernel object in kernel memory, and the granted access mask.

Once an application has finished with a handle, it can be closed using the `NtClose` API. If you've received a PowerShell object from a `Get` or `New` call, then you can call the `Close` method on the object to close the handle. You can also close an object handle automatically in PowerShell by using the `Use-NtObject` command to invoke a script block that closes the handle once it finishes executing. [Listing 2-16](#) provides examples of both approaches.

```
PS> $m = New-NtMutant \BaseNamedObjects\ABC
PS> $m.IsClosed
False

PS> $m.Close()
PS> $m.IsClosed
True
```

```
PS> Use-NtObject($m = New-NtMutant \BaseNamedObjects\ABC) {  
    $m.FullPath  
}  
\BaseNamedObjects\ABC  
  
PS> $m.IsClosed  
True
```

Listing 2-16: Closing an object handle

If you do not close handles manually, the .NET garbage collector will close them automatically for objects that are not referenced (for example, held in a PowerShell variable). You should get into the habit of manually closing handles, though; otherwise, you might have to wait a long time for the resources to be released, as the garbage collector could run at any time.

If the kernel object structure is no longer referenced, either through a handle or by a kernel component, then the object will also be destroyed. Once an object is destroyed, all its allocated memory is cleaned up and, if it exists, its name in the OMNS is removed.

PERMANENT OBJECTS

It is possible to get the kernel to mark an object as permanent, preventing the object from being destroyed when all handles close and allowing its name to remain in the OMNS. To make an object permanent, you need to either specify the `Permanent` attribute flag when creating the object or use the system call `NtMakePermanentObject`, which is mapped to the `MakePermanent` call on any object handle returned by a `Get` or `New` command. You need a special privilege, `SeCreatePermanentPrivilege`, to do this; we'll discuss privileges in [Chapter 4](#).

The reverse operation, `NtMakeTemporaryObject` (or the `MakeTemporary` method in PowerShell), removes the permanent setting and allows an object to be destroyed. The destruction won't happen until all handles to the object have closed. This operation doesn't require any special privileges, but it does require `Delete` access on the object to succeed.

Note that `File` and `Key` objects always have permanent names as they don't exist in the OMNS; to remove the names for these types of objects, you must use a system call to explicitly delete them.

Handle Duplication

You can duplicate handles using the `NtDuplicateObject` system call. The primary reason you might want to do this is to allow a process to take an additional reference to a kernel object. The kernel object won't be destroyed until all handles to it are closed, so creating a new handle maintains the kernel object.

Handle duplication can additionally be used to transfer handles between processes if the source and destination process handles have `DupHandle` access. You can also use handle duplication to reduce the access rights on a handle. For example, when you pass a file handle to a new process, you could grant the duplicated handle only read access, preventing the new process from writing to the object. However, you should not rely on this approach for reducing the handle's granted access; if the process with the handle has access to the resource, it can just reopen it to get write access.

Listing 2-17 shows some examples of using the `Copy-NtObject` command, which wraps `NtDuplicateObject`, to perform some duplication in the same process. We'll come back to handle duplication and security checks in **Chapter 8**.

```
❶ PS> $mut = New-NtMutant "\BaseNamedObjects\ABC"
PS> $mut.GrantedAccess
ModifyState, Delete, ReadControl, WriteDac, WriteOwner, Synchronize

❷ PS> Use-NtObject($dup = Copy-NtObject $mut) {
    $mut
    $dup
    Compare-NtObject $mut $dup
}
Handle Name NtTypeName Inherit ProtectFromClose
-----
1616 ABC Mutant False False
2212 ABC Mutant False False
True

❸ PS> $mask = Get-NtAccessMask -MutantAccess ModifyState
PS> Use-NtObject($dup = Copy-NtObject $mut -DesiredAccessMask $mask) {
    $dup.GrantedAccess
    Compare-NtObject $mut $dup
}
ModifyState
True
```

Listing 2-17: Using Copy-NtObject to duplicate handles

First, we create a new `Mutant` object to test handle duplication and extract the current granted access, which shows six access rights ❶. For the initial duplication, we'll keep the same granted access ❷. You can see in the first column of the output that the handles are different. However, our call to `Compare-NtObject` to determine whether the two handles refer to the same underlying kernel object returns `True`. Next, we get an access mask for `Mutant` `ModifyState` access and duplicate the handle, requesting that access ❸. We can see in the output that the granted access is now only `ModifyState`. However, the `Compare-NtObject` return value still indicates the handles refer to the same object.

Also relevant to handle duplication are the handle attributes `Inherit` and `ProtectFromClose`. Setting `Inherit` allows a new process to inherit the handle when it's created. This allows you to pass handles to a new process to perform tasks such as redirecting console output text to a file.

Setting `ProtectFromClose` protects the handle from being closed. You can set this attribute by setting the `ProtectFromClose` property on the object, which will set the attribute on the native handle. [Listing 2-18](#) shows an example of its use.

```
PS> $mut = New-NtMutant
PS> $mut.ProtectFromClose = $true
PS> Close-NtObject -SafeHandle $mut.Handle -CurrentProcess
STATUS_HANDLE_NOT_CLOSABLE
```

Listing 2-18: Testing the ProtectFromClose handle attribute

Any attempt to close the handle will fail with a `STATUS_HANDLE_NOT_CLOSABLE` status code, and the handle will stay open.

Query and Set Information System Calls

A kernel object typically stores information about its state. For example, a `Process` object stores a timestamp of when it was created. To allow us to retrieve this information, the kernel could have implemented a specific “get process creation time” system call. However, due to the volume of information stored for the various types of objects, this approach would quickly become unworkable.

Instead, the kernel implements generic `Query` and `Set` information system calls whose parameters follow a common pattern for all kernel object types. [Listing 2-19](#) shows the `Query` information system call's pattern, using the `Process` type as an example; for other types, just replace `Process` with the name of the kernel type.

```
NTSTATUS NtQueryInformationProcess(  
    HANDLE                Handle,  
    PROCESS_INFORMATION_CLASS InformationClass,  
    PVOID                 Information,  
    ULONG                 InformationLength,  
    PULONG                 ReturnLength  
)
```

Listing 2-19: An example Query information system call for the Process type

All `Query` information system calls take an object handle as the first parameter. The second parameter, `InformationClass`, describes the type of process information to query. The information class is an enumerated value; the SDK specifies the names of the information classes, which we can extract and implement in PowerShell. Querying certain kinds of information might require special privileges or administrator access.

For every information class, we need to specify an opaque buffer to receive the queried information, as well as the length of the buffer. The system call also returns a length value, which serves two purposes: it indicates how much of the buffer was populated if the system call was successful, and if the system call failed, it indicates how big the buffer needs to be with `STATUS_INFO_LENGTH_MISMATCH` or `STATUS_BUFFER_TOO_SMALL`.

You should be careful about relying on the returned length to determine how big a buffer to pass to the query, however. Some information classes and types do not correctly set the length needed if you supply a buffer that is too small. This makes it difficult to query data without knowing its format in advance. Unfortunately, even the SDK rarely documents the exact sizes required.

As shown in [Listing 2-20](#), the `Set` information call follows a similar pattern. The main differences are that there's no return length parameter, and in this case the buffer is an input to the system call rather than an output.

```
NTSTATUS NtSetInformationProcess(
    HANDLE                Handle,
    PROCESS_INFORMATION_CLASS InformationClass,
    PVOID                 Information,
    ULONG                 InformationLength
)
```

Listing 2-20: An example Set information system call for the Process type

In the PowerShell module, you can query a type's information class names using the `Get-NtObjectInformationClass` command, as shown in [Listing 2-21](#). Bear in mind that some information class names might be missing from the list, as Microsoft doesn't always document them.

```
PS> Get-NtObjectInformationClass Process
Key                                Value
---                                -
ProcessBasicInformation           0
ProcessQuotaLimits                1
ProcessIoCounters                 2
ProcessVmCounters                 3
ProcessTimes                      4
--snip--
```

Listing 2-21: Listing the information classes for the Process type

To call the query information system call, use `Get-NtObjectInformation`, specifying an open object handle and the information class. To call `SetInformation`, use `Set-NtObjectInformation`. [Listing 2-22](#) shows an example of how to use `Get-NtObjectInformation`.

```
PS> $proc = Get-NtProcess -Current
❶ PS> Get-NtObjectInformation $proc ProcessTimes
Get-NtObjectInformation : (0xC0000023) - {Buffer Too Small}
The buffer is too small to contain the entry. No information has been written to the
--snip--

❷ PS> Get-NtObjectInformation $proc ProcessTimes -Length 32
43
231
39
138
--snip--

❸ PS> Get-NtObjectInformation $proc ProcessTimes -AsObject
```

CreateTime	ExitTime	KernelTime	UserTime
-----	-----	-----	-----
132480295787554603	0	35937500	85312500

Listing 2-22: Querying a Process object for basic information

The `Process` type doesn't set the return length for the `ProcessTimes` information class, so if you don't specify any length, the operation generates a `STATUS_BUFFER_TOO_SMALL` error ❶. However, through inspection or brute force, you can discover that the length of the data is 32 bytes. Specifying this value using the `Length` parameter ❷ allows the query to succeed and return the data as an array of bytes.

For many information classes, the `Get-NtObjectInformation` command knows the size and structure of the query data. If you specify the `AsObject` parameter ❸, you can get a preformatted object rather than an array of bytes.

Also, for many information classes the handle object already exposes properties and methods to set or query values. The values will be decoded into a usage format; for example, in [Listing 2-22](#), the times are in an internal format. The `CreationTime` property on the object will take this internal format and convert it to a human-readable date and time.

You can easily inspect properties by accessing them on the object or using the `Format-List` command. For example, [Listing 2-23](#) lists all the properties on a `Process` object, then queries for the formatted `CreationTime`.

```
PS> $proc | Format-List
SessionId      : 2
ProcessId      : 5484
ParentProcessId : 8108
PebAddress     : 46725963776
--snip--

PS> $proc.CreationTime
Saturday, October 24, 17:12:58
```

Listing 2-23: Querying a handle object for properties and inspecting the CreationTime

The `QueryInformation` and `SetInformation` classes for a type typically have the same enumerated values. The kernel can restrict the information class's enumerated values to one type of operation, returning the `STATUS_INVALID`

`_INFO_CLASS` status code if it's not a valid value. For some types, such as registry keys, the information class differs between querying and setting, as you can see in [Listing 2-24](#).

```
PS> Get-NtObjectInformationClass Key
Key                               Value
---                               -
KeyBasicInformation              0
--snip--

PS> Get-NtObjectInformationClass Key -Set
Key                               Value
---                               -
KeyWriteTimeInformation         0
--snip--
```

Listing 2-24: Inspecting the `QueryInformation` and `SetInformation` classes for the `Key` type

Calling `Get-NtObjectInformationClass` with just the type name returns the `QueryInformation` class. If you specify the type name and the `Set` parameter, you get the `SetInformation` class. Notice how the two entries shown have different names and therefore represent different information.

The Input/Output Manager

The input/output (I/O) manager provides access to I/O devices through *device drivers*. The primary purpose of these drivers is to implement a filesystem. For example, when you open a document on your computer, the file is made available through a filesystem driver. The I/O manager supports other kinds of drivers, for devices such as keyboards and video cards, but these other drivers are really just filesystem drivers in disguise.

You can manually load a new driver through the `NtLoadDriver` system call or do so automatically using the Plug and Play (PnP) manager. For every driver, the I/O manager creates an entry in the *Driver* directory. You can list the contents of this directory only if you're an administrator. Fortunately, as a normal user, you don't need to access anything in the *Driver* directory. Instead, you can interact with the driver through a *Device* object, normally created in the *Device* directory.

Drivers are responsible for creating new *Device* objects using the `IoCreateDevice` API. A driver can have more than one *Device* object associ-

ated with it; it may also have zero associated `Device` objects if it doesn't require user interaction. As [Listing 2-25](#) shows, we can list the contents of the `Device` directory as a normal user through the OMNS.

```
PS> ls NtObject:\Device
Name                                     TypeName
----                                     -
_HID00000034                           Device
DBUtil_2_3                             Device
000000c7                               Device
000000b3                               Device
UMDFCtrlDev-0f8ff736-55d7-11ea-b5d8-2... Device
0000006a                               Device
--snip--
```

Listing 2-25: Displaying the Device objects

In the output, we can see that the objects' type names are all `Device`. However, if you go looking for a system call with `Device` in the name, you'll come up empty. That's because we don't interact with the I/O manager using dedicated system calls; rather, we use `File` object system calls such as `NtCreateFile`. We can access these system calls through `New-NtFile` and `Get-NtFile`, which create and open files, respectively, as shown in [Listing 2-26](#).

```
PS> Use-NtObject($f = Get-NtFile "\SystemRoot\notepad.exe") {
    $f | Select-Object FullPath, NtTypeName
}
FullPath                                     NtTypeName
-----
❶ \Device\HarddiskVolume3\Windows\notepad.exe File

PS> Get-Item NtObject:\Device\HarddiskVolume3
Name          TypeName
----          -
HarddiskVolume3 Device
```

Listing 2-26: Opening a device object and displaying its volume path

In this example, we open `notepad.exe` from the `Windows` directory. The `SystemRoot` symbolic link points to the `Windows` directory on the system drive. As the `SystemRoot` symbolic link is part of the OMNS, the OMNS initially handles file access. With an open handle, we can select the full path to the file and the type name.

Looking at the result, we can see that the full path starts with *Device\HarddiskVolume3*, followed by *Windows\notepad.exe* ❶. If we try to display the device, we find it's of type *Device*. Once the object manager finds the *Device* object, it hands off responsibility for the rest of the path to the I/O manager, which calls an appropriate method inside the kernel driver.

We can list the drivers loaded into the kernel using the `Get-NtKernelModule` command ([Listing 2-27](#)).

```
PS> Get-NtKernelModule
Name           ImageBase      ImageSize
----           -
ntoskrnl.exe   FFFFFFF8053BEA000 11231232
hal.dll        FFFFFFF8053BE0700 667648
kd.dll         FFFFFFF8053B42E00 45056
msrpc.sys      FFFFFFF8053B48E00 393216
ksecdd.sys     FFFFFFF8053B45E00 172032
--snip--
```

Listing 2-27: Enumerating all loaded kernel drivers

Unlike other operating systems, such as Linux, Windows does not implement core network protocols like TCP/IP using built-in system calls. Instead, Windows has an I/O manager driver, the *Ancillary Function Driver (AFD)*, which provides access to networking services for an application. You don't need to deal with the driver directly; Win32 provides a BSD sockets-style API, called *WinSock*, to handle access to it. In addition to the standard internet protocol suite, such as TCP/IP, AFD also implements other network socket types, such as Unix sockets and bespoke Hyper-V sockets for communication with virtual machines.

That's all we'll say for now about the I/O manager. Next, let's turn to another important subsystem, the process and thread manager.

The Process and Thread Manager

All user-mode code lives in the context of a *process*, each of which has one or more *threads* that control the execution of the code. Processes and threads are both securable resources. This makes sense: if you could access a process, you could modify its code and execute it in the context of a different user identity. So, unlike most other kernel objects, you can't

open a process or thread by name. Instead, you must open them via a unique, numeric *process ID (PID)* or *thread ID (TID)*.

To get a list of running processes and threads you could brute-force the ID space by calling the open system call with every possible ID, but that would take a while. Fortunately, the `NtQuerySystemInformation` system call provides the `SystemProcessInformation` information class, which lets us enumerate processes and threads without having access to the `Process` object.

We can access the list of processes and threads by using the `Get-NtProcess` and `Get-NtThread` commands and passing them the `InfoOnly` parameter, as shown in [Listing 2-28](#). We can also use the built-in `Get-Process` command to produce a similar output. Each of the returned objects has a `Threads` property that we can query for the thread information.

```
PS> Get-NtProcess -InfoOnly
PID PPID Name          SessionId
--- --
0   0   Idle              0
4   0   System            0
128 4   Secure System    0
192 4   Registry          0
812 4   smss.exe          0
920 892 csrss.exe         0
--snip--

PS> Get-NtThread -InfoOnly
TID PID ProcessName StartAddress
--- --
0   0   Idle          FFFFFFFF8004C9CAFD0
0   0   Idle          FFFFFFFF8004C9CAFD0
--snip--
```

Listing 2-28: Displaying processes and threads without high privilege

The first two processes listed in the output are special. The first is the `Idle` process, with PID `0`. This process contains threads that execute when the operating system is idle, hence its name. It's not a process you'll need to deal with regularly. The `System` process, with PID `4`, is important because it runs entirely in kernel mode. When the kernel or a driver needs to execute a background thread, the thread is associated with the `System` process.

To open a process or thread, we can pass `Get-NtProcess` OR `Get-NtThread` the PID or TID we want to open. The command will return a `Process` OR `Thread` object that we can then interact with. For example, [Listing 2-29](#) shows

how to query the command line and executable path of the current process.

```
PS> $proc = Get-NtProcess -ProcessId $pid
PS> $proc.CommandLine
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"

PS> $proc.Win32ImagePath
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

Listing 2-29: Opening the current process by its process ID

When you open a `Process` or `Thread` object using its ID, you'll receive a handle. For convenience, the kernel also supports two *pseudo handles* that refer to the current process and the current thread. The current process's pseudo handle is the value `-1` converted to a handle, and for the current thread, it's `-2`. You can access these pseudo handles by passing the `Current` parameter instead of an ID to the `Get-NtProcess` and `Get-NtThread` commands.

Note that the security of a process and its threads is independent. If you know the ID of a thread, it's possible to access the thread handle inside a process even if you can't access the process itself.

The Memory Manager

Every process has its own virtual memory address space for a developer to use as they see fit. A 32-bit process can access up to 2GB of virtual memory address space (4GB on 64-bit Windows), while a 64-bit process can access up to 128TB. The kernel's *memory manager* subsystem controls the allocation of this address space.

You're unlikely to have 128TB of physical memory in your computer, but the memory manager has ways of making it look like you have more physical memory than you do. For example, it can use a dedicated file on your filesystem, called a *pagefile*, to temporarily store memory when it's not currently needed. As your filesystem's available storage space is much larger than your computer's physical memory, this can provide the appearance of a large amount of memory.

The virtual memory space is shared by memory allocations, and it stores each process's running state as well as its executable code. Each memory allocation can have a range of protection states, such as `ReadOnly` or

ReadWrite, which must be set according to the memory's purpose. For example, for code to be executed, the memory must have a protection state Of ExecuteRead Or ExecuteReadWrite.

You can query all memory status information for a process by calling NtQueryVirtualMemory, if you have the QueryLimitedInformation access right on the process handle. However, reading and writing the memory data requires the VmRead and VmWrite access rights, respectively, and a call to NtReadVirtualMemory and NtWriteVirtualMemory.

It's possible to allocate new memory and free memory in a process using NtAllocateVirtualMemory and NtFreeVirtualMemory, which both require the VmOperation access right. Finally, you can change the protection on memory using NtProtectVirtualMemory, which also requires VmOperation access.

NtVirtualMemory Commands

PowerShell wraps these system calls using the Get-, Add-, Read-, Write-, Remove-, and Set-NtVirtualMemory commands. Note that these commands all accept an optional Process parameter that lets you access memory in a different process from the current one. [Listing 2-30](#) shows the commands in action.

```

❶ PS> Get-NtVirtualMemory
Address          Size    Protect      Type    State    Name
-----
000000007FFE0000 4096    ReadOnly     Private Commit
000000007FFE0000 4096    ReadOnly     Private Commit
000000E706390000 241664  None         Private Reserve
000000E7063CB000 12288   ReadWrite, Guard Private Commit
000000E7063CE000 8192    ReadWrite     Private Commit
000000F6583F0000 12288   ReadOnly     Mapped   Commit   powershell.exe.mui
--snip--

❷ PS> $addr = Add-NtVirtualMemory -Size 1000 -Protection ReadWrite
PS> Get-NtVirtualMemory -Address $addr
Address          Size    Protect      Type    State    Name
-----
000002624A440000 4096    ReadWrite     Private Commit

❸ PS> Read-NtVirtualMemory -Address $addr -Size 4 | Out-HexDump
00 00 00 00

❹ PS> Write-NtVirtualMemory -Address $addr -Data @(1,2,3,4)
4

```

```

❶ PS> Read-NtVirtualMemory -Address $addr -Size 4 | Out-HexDump
01 02 03 04

❷ PS> Set-NtVirtualMemory -Address $addr -Protection ExecuteRead -Size 4
ReadWrite

❸ PS> Get-NtVirtualMemory -Address $addr
Address          Size    Protect          Type    State    Name
-----
000002624A440000 4096    ExecuteRead      Private Commit

❹ PS> Remove-NtVirtualMemory -Address $addr
PS> Get-NtVirtualMemory -Address $addr
Address          Size    Protect          Type    State    Name
-----
000002624A440000 196608 NoAccess          None    Free

```

Listing 2-30: Performing various memory operations on a process

Here, we perform several operations. First we use `Get-NtVirtualMemory` to list all the memory regions being used by the current process ❶. The returned list will be large, but the excerpt shown here should give you a rough idea of how the information is presented. It includes the address of the memory region, its size, its protection, and its state. There are three possible state values:

Commit Indicates that the virtual memory region is allocated and available for use.

Reserve Indicates that the virtual memory region has been allocated but there is currently no backing memory. Using a reserved memory region will cause a crash.

Free Indicates that the virtual memory region is unused. Using a free memory region will cause a crash.

You may wonder what the difference is between `Reserve` and `Free`, if using both reserved and free memory regions will cause a crash. The `Reserve` state allows you to reserve virtual memory regions for later use so that nothing else can allocate memory within that range of memory addresses. You can later convert the `Reserve` state to `Commit` by re-calling `NtAllocateVirtualMemory`. The `Free` state indicates regions freely available for allocation. We'll cover what the `Type` and `Name` columns indicate later in this section.

Next, we allocate a 1,000-byte read/write region and capture the address in a variable ❷. Passing the address to `Get-NtVirtualMemory` allows us to query only that specific virtual memory region. You might notice that although we requested a 1,000-byte region, the size of the region returned is 4,096 bytes. This is because all virtual memory allocations on Windows have a minimum allocation size; on the system I'm using, the minimum is 4,096 bytes. It's therefore not possible to allocate a smaller region. For this reason, these system calls are not particularly useful for general program allocations; rather, they're primitives on which "heap" memory managers are built, such as `malloc` from the C library.

Next, we read and write to the memory region we just allocated. First we use `Read-NtVirtualMemory` to read out 4 bytes of the memory region and find that the bytes are all zeros ❸. Next, we write the bytes 1, 2, 3, and 4 to the memory region using `Write-NtVirtualMemory` ❹. We read the bytes to confirm that the write operation succeeded ❺; the two values should match, as shown in the output.

With the memory allocated, we can change the protection using `Set -NtVirtualMemory`. In this case, we make the allocated memory executable by specifying the protection as `ExecuteRead` ❻. Querying the current state of the memory region using the `Get-NtVirtualMemory` command ❼ shows that the protection has changed from `ReadWrite` to `ExecuteRead`. Also notice that although we requested to change the protection of only 4 bytes, the entire 4,096-byte region is now executable. This is again due to the minimum memory allocation size.

Finally, we free the memory using `Remove-NtVirtualMemory` and verify that the memory is now in the `Free` state ❽. Memory allocated using `NtAllocateVirtualMemory` is considered private, as indicated by the value of the `Type` property shown in [Listing 2-30](#).

Section Objects

Another way of allocating virtual memory is through `Section` objects. A `Section` object is a kernel type that implements memory-mapped files. We can use `Section` objects for two related purposes:

- Reading or writing a file as if it were all read into memory
- Sharing memory between processes so that the modification in one process is reflected in the other

We can create a `Section` object via the `NtCreateSection` system call or the `New-NtSection` PowerShell command. We must specify the size of the mapping, the protection for the memory, and an optional file handle; in return, we get a handle to the section.

However, creating a section doesn't automatically allow us to access the memory; we first need to map it into the virtual memory address space using `NtMapViewOfSection` OR `Add-NtSection`. **Listing 2-31** provides an example in which we create an anonymous section and map it into memory.

```

❶ PS> $s = New-NtSection -Size 4096 -Protection ReadWrite
❷ PS> $m = Add-NtSection -Section $s -Protection ReadWrite
PS> Get-NtVirtualMemory $m.BaseAddress
Address          Size Protect   Type   State   Name
-----
000001C3DD0E0000 4096 ReadWrite Mapped Commit

❸ PS> Remove-NtSection -Mapping $m
PS> Get-NtVirtualMemory -Address 0x1C3DD0E0000
Address          Size Protect   Type   State   Name
-----
000001C3DD0E0000 4096 NoAccess  None   Free

❹ PS> Add-NtSection -Section $s -Protection ExecuteRead
Exception calling "Map" with "9" argument(s):
"(0xC000004E) - A view to a section specifies a protection which is
incompatible with the initial view's protection."

```

Listing 2-31: Creating a section and mapping it into memory

To start, we create a `Section` object with a size of 4,096 bytes and protection of `ReadWrite` ❶. We don't specify a `File` parameter, which means it's anonymous and not backed by any file. If we gave the `Section` object an OMNS path, the anonymous memory it represents could be shared with other processes.

We then map the section into memory using `Add-NtSection`, specifying the protection we want for the memory, and query the mapped address to verify that the operation succeeded ❷. Note that the `Type` is set to `Mapped`. When we're done with the mapping, we call `Remove-NtSection` to unmap the section and then verify that it's now free ❸.

Finally, we demonstrate that we can't map a section with different protection than that granted when we created the `Section` object ❹. When we try

to map the section with read and execute permissions, which aren't compatible, we see an exception.

The protection you're allowed to use to map a section object into memory depends on two things. The first is the protection specified when the section object was created. For example, if the section was created with `ReadOnly` protection, you can never map it to be writable.

The second dependency is the access granted to the section handle you're mapping. If you want to map the section as readable, then the handle must have `MapRead` access. To map it to be writable, you need both `MapRead` and `MapWrite` access. (And, of course, having just `MapWrite` access isn't sufficient to map the section as writable if the original section object was not specified with a writable protection.)

It's possible to map a section into another process by specifying a process handle to `Add-NtSection`. We don't need to specify the process to `Remove-NtSection`, as the mapping object knows what process it was mapped in. In the memory information output, the `Name` column would be populated by the name of the backing file, if it exists.

The section we created was anonymous, so we don't see anything in the `Name` column, but we can perform a query to find mapped sections that are backed by files using the command shown in [Listing 2-32](#).

```
PS> Get-NtVirtualMemory -Type Mapped | Where-Object Name -ne ""
Address      Size      Protect  Type    State  Name
-----
000001760DB90000 815104  ReadOnly Mapped  Commit locale.nls
000001760DC60000 12288   ReadOnly Mapped  Commit powershell.exe.mui
000001760DEE0000 20480   ReadOnly Mapped  Commit winnlsres.dll
000001760F720000 3371008 ReadOnly Mapped  Commit SortDefault.nls
--snip--
```

Listing 2-32: Listing mapped files with names

In addition to the `Anonymous` and `Mapped` types, there is a third section type, the `Image` type. When provided with a file handle to a Windows executable, the kernel will automatically parse the format and generate multiple subsections that represent the various components of the executable. To create a mapped image from a file, we need only `Execute` access on the file handle; the file doesn't need to be readable for us.

Windows uses image sections extensively to simplify the mapping of executables into memory. We can specify an image section by passing the Image flag when creating the Section object or by using the `New-NtSectionImage` command, as shown in [Listing 2-33](#).

```

PS> $sect = New-NtSectionImage -Win32Path "C:\Windows\notepad.exe"
❶ PS> $map = Add-NtSection -Section $sect -Protection ReadOnly
PS> Get-NtVirtualMemory -Address $map.BaseAddress
Address          Size    Protect    Type    State    Name
-----
❷ 00007FF667150000 4096    ReadOnly    Image    Commit    notepad.exe

❸ PS> Get-NtVirtualMemory -Type Image -Name "notepad.exe"
Address          Size    Protect    Type    State    Name
-----
00007FF667150000 4096    ReadOnly    Image    Commit    notepad.exe
00007FF667151000 135168 ExecuteRead Image    Commit    notepad.exe
00007FF667172000 36864    ReadOnly    Image    Commit    notepad.exe
00007FF66717B000 12288    WriteCopy    Image    Commit    notepad.exe
00007FF66717E000 4096    ReadOnly    Image    Commit    notepad.exe
00007FF66717F000 4096    WriteCopy    Image    Commit    notepad.exe
00007FF667180000 8192    ReadOnly    Image    Commit    notepad.exe

❹ PS> Out-HexDump -Buffer $map -ShowAscii -Length 128
4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 - MZ.....
B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 - .....@.....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 - .....
00 00 00 00 00 00 00 00 00 00 00 00 00 F8 00 00 00 - .....
0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 - .....!.L.!Th
69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F - is program canno
74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 - t be run in DOS
6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00 - mode....$.....

```

Listing 2-33: Mapping notepad.exe and viewing the loaded image

As you can see, we don't need to specify `ExecuteRead` or `ExecuteReadWrite` protection when mapping the image section. Any protection, including `ReadOnly`, will work ❶. When we get the memory information for a map-based address, we see that there is no executable memory there and that the allocation is only 4,096 bytes ❷, which seems far too small for *notepad.exe*. This is because the section is made up of multiple smaller mapped regions. If we filter out the memory information for the mapped name ❸, we can see the executable memory. Using the `Out-HexDump` command, we can print the contents of the mapped file buffer ❹.

Code Integrity

One important security task is ensuring that the code running on your computer is the same code the manufacturer intended you to run. If a malicious user has modified operating system files, you might encounter security issues such as the leaking of private data.

Microsoft considers the integrity of code running on Windows to be so important that there is an entire subsystem to deal with it. This *code integrity* subsystem verifies and restricts what files can execute in the kernel, and optionally in user mode, by checking the code's integrity. The memory manager can consult with the code integrity subsystem when it loads an image file if it needs to check whether the executable is correctly signed.

Almost every executable on a default Windows installation is signed using a mechanism called *Authenticode*. This mechanism allows a cryptographic signature to be embedded in the executable file or collected inside a catalog file. The code integrity subsystem can read this signature, verify that it's valid, and make trust decisions based on it.

We can use the `Get-AuthenticodeSignature` command to query the signing status of an executable, as shown in [Listing 2-34](#).

```
PS> Get-AuthenticodeSignature "$env:WinDir\system32\notepad.exe" | Format-List
SignerCertificate : [Subject]
                  CN=Microsoft Windows, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
--snip--
Status            : Valid
StatusMessage     : Signature verified.
Path              : C:\WINDOWS\system32\notepad.exe
SignatureType     : Catalog
IsOSBinary        : True
```

Listing 2-34: Displaying the Authenticode signature for a kernel driver

Here, we query the signing status of the *notepad.exe* executable file, formatting the command's output as a list. The output starts with information about the signer's X.509 certificate. Here, I've shown only the subject name, which clearly indicates that this file is signed by Microsoft.

Next is the status of the signature; in this case, the status indicates that the file is valid and that the signature has been verified. It's possible to have a signed file whose signature is invalid; for example, when the cer-

tificate has been revoked. In that case, the status is likely to show an error, such as `NotSigned`.

The `SignatureType` property shows that this signature was based on a catalog file rather than being embedded in the file. We can also see that this file is an operating system binary, as determined by information embedded in the signature.

The most common trust decision the code integrity subsystem makes is checking whether a kernel driver can load. Each driver file must have a signature that derives its trust from a Microsoft-issued key. If the signature is invalid or doesn't derive from a Microsoft-issued key, then the kernel can block loading of the driver to preserve system integrity.

Advanced Local Procedure Call

The *advanced local procedure call (ALPC)* subsystem implements local, cross-process communication. To use ALPC, you must first create a server ALPC port using the `NtCreateAlpcPort` system call and specify a name for it inside the OMNS. A client can then use this name by calling the `NtConnectAlpcPort` system call to connect to the server port.

At a basic level, the ALPC port allows the secure transmission of discrete messages between a server and a client. ALPC provides the underlying transport for local remote procedure call APIs implemented in Windows.

The Configuration Manager

The *configuration manager*, known more commonly as the *registry*, is an important component for configuring the operating system. It stores a variety of configuration information, ranging from the system-critical list of available I/O manager device drivers to the (less critical) last position on the screen of your text editor's window.

You can think of the registry as a filesystem in which *keys* are like folders and *values* are like files. You can access it through the OMNS, although you must use registry-specific system calls. The root of the registry is the OMNS path `REGISTRY`. You can list the registry in PowerShell using the `NtObject` drive, as shown in [Listing 2-35](#).

```
PS> ls NtObject:\REGISTRY
Name      TypeName
----
```


A	Key
MACHINE	Key
USER	Key
WC	Key

Listing 2-35: Enumerating the registry root key

You can replace `NtObject:\REGISTRY` in [Listing 2-35](#) with `NtKey:\` to make accessing the registry simpler.

The kernel pre-creates the four keys shown here when it initializes. Each of the keys is a special *attachment point* at which you can attach a registry hive. A *hive* is a hierarchy of `Key` objects underneath a single root key. An administrator can load new hives from a file and attach them to these preexisting keys.

Note that PowerShell already comes with a drive provider that you can use to access the registry. However, this drive provider exposes only the Win32 view of the registry, which hides the internal details about the registry from view. We'll cover the Win32 view of the registry separately in [Chapter 3](#).

You can interact with the registry directly, using the `Get-NtKey` and `New-NtKey` commands to open and create `Key` objects, respectively. You can also use `Get-NtKeyValue` and `Set-NtKeyValue` to get and set key values. To remove keys or values, use `Remove-NtKey` or `Remove-NtKeyValue`. [Listing 2-36](#) shows a few of these commands in action.

```
PS> $key = Get-NtKey \Registry\Machine\SOFTWARE\Microsoft\.NETFramework
PS> Get-NtKeyValue -Key $key
```

Name	Type	DataObject
-----	----	-----
Enable64Bit	Dword	1
InstallRoot	String	C:\Windows\Microsoft.NET\Framework64\
UseRyuJIT	Dword	1
DbgManagedDebugger	String	"C:\Windows\system32\vsjitdebugger.exe"...
DbgJITDebugLaunchSetting	Dword	16

Listing 2-36: Opening a registry key and querying its values

We open a `Key` object using the `Get-NtKey` command. We can then query the values stored in the `Key` object using the `Get-NtKeyValue` command. Each entry in the output shows the name of the value, the type of data stored, and a string representation of the data.

Worked Examples

Using PowerShell, you can easily change this book's example scripts to do many different things. To encourage experimentation, each chapter wraps up with a set of worked examples repurposing the various commands you've learned.

In these examples, I'll also highlight times where I've discovered security vulnerabilities using this tooling. This should give you a clear indication of what to look for in Microsoft or third-party applications if you're a security researcher; likewise, for developers, it will help you avoid certain pitfalls.

Finding Open Handles by Name

The objects returned by the `Get-NtHandle` command have additional properties that allow you to query the object's name and security descriptor. These properties are not shown by default, as they're expensive to look up; doing so requires opening the process containing the handle for `DupHandle` access, duplicating the handle back to the calling PowerShell instance, and finally querying the property.

If performance doesn't matter to you, then you can use the code in

Listing 2-37 to find all open files matching a specific filename.

```
PS> $hs = Get-NtHandle -ObjectType File | Where-Object Name -Match Windows
PS> $hs | Select-Object ProcessId, Handle, Name
ProcessId Handle Name
-----
3140      64  \Device\HarddiskVolume3\Windows\System32
3140     1628 \Device\HarddiskVolume3\Windows\System32\en-US\KernelBase.dll.mui
3428      72  \Device\HarddiskVolume3\Windows\System32
3428     304  \Device\HarddiskVolume3\Windows\System32\en-US\svchost.exe.mui
3428     840  \Device\HarddiskVolume3\Windows\System32\en-US\crypt32.dll.mui
3428    1604  \Device\HarddiskVolume3\Windows\System32\en-US\winlnsres.dll.mui
--snip--
```

Listing 2-37: Finding File object handles that match a specific name

This script queries for all `File` object handles and filters them to only the ones with the string `Windows` in the `Name` property, which represents the filepath. Once the `Name` property has been queried, it's cached so you can then display it to the console with a custom selection.

Note that because it duplicates the handle from the process, this script can only show handles in processes the caller can open. To get the best results, run it as an administrator user who can open the maximum number of processes.

Finding Shared Objects

When you query the list of handles using the `Get-NtHandle` command, you also get the address of the object in kernel memory. When you open the same kernel object, you'll get different handles, but they will still point to the same kernel object address.

You can use the object address to find processes that share handles. This can be interesting for security in cases where an object is shared between two processes with different privileges. The lower-privileged process might be able to modify the properties of the object to bypass security checks in the higher-privileged process, enabling it to gain additional privileges.

In fact, I used this technique to find security issue CVE-2019-0943 in Windows. At the root of the issue was a privileged process, the Windows Font Cache, that shared section handles with a low-privileged process. The low-privileged process could map the shared section to be writable and modify contents that the privileged process assumed couldn't be modified. This effectively allowed the low-privileged process to modify arbitrary memory in the privileged process, resulting in privileged code execution.

Listing 2-38 gives an example of finding writable `Section` objects shared between two processes.

```
PS> $ss = Get-NtHandle -ObjectType Section -GroupByAddress |  
Where-Object ShareCount -eq 2  
PS> $mask = Get-NtAccessMask -SectionAccess MapWrite  
PS> $ss = $ss | Where-Object {Test-NtAccessMask $_.AccessIntersection $mask}  
PS> foreach($s in $ss) {  
    $count = ($s.ProcessIds | Where-Object {  
        Test-NtProcess -ProcessId $_ -Access DupHandle  
    }).Count  
    if ($count -eq 1) {  
        $s.Handles | Select ProcessId, ProcessName, Handle  
    }  
}  
ProcessId ProcessName Handle
```

```
-----  
9100 Chrome.exe      4400  
4072 audiodg.exe     2560
```

Listing 2-38: Finding shared Section handles

We first get the handles, specifying the `GroupByAddress` parameter. This returns a list of groups organized based on the kernel object address, instead of a list of handles. You can also group handles using the built-in `Group-Object` command; however, the groups returned by `GroupByAddress` have additional properties, including `ShareCount`, which indicates the number of unique processes an object is shared with. Here, we filter to include only handles that are shared between two processes.

Next, we want to find `Section` objects that can be mapped as writable. We first check that all the handles have `MapWrite` access. As mentioned earlier, the `Section` object's protection must also be writable for us to be able to map it as writable. Oddly, we can't query for the original protection that was assigned when the `Section` object was created, but checking for `MapWrite` access is a simple proxy. We use the `AccessIntersection` property, which contains the granted access rights shared among all the handles.

Now that we have potential candidates for shared sections, we need to work out which meet the criterion that we can access only one of the processes containing the section handle. We're making another assumption here: if we can open only one of the two processes that share the handle for `DupHandle` access, then we've got a section shared between a privileged and a low-privileged process. After all, if you had `DupHandle` access to both processes, you could already compromise the processes by stealing all their handles or duplicating their process handles, and if you couldn't get `DupHandle` access to either process, then you couldn't get access to the section handle at all.

The result shown in [Listing 2-38](#) is a section shared between Chrome and the Audio Device Graph process. The shared section is used to play audio from the browser, and it's probably not a security issue. However, if you run the script on your own system, you might find shared sections that are.

Note that once the `Section` object is mapped into memory, the handle is no longer required. Therefore, you might miss some shared sections that were mapped when the original handle closed. It's also highly likely you'll get false positives, such as `Section` objects that are intentionally writable

by everyone. The goal here is to find a potential attack surface on Windows. You must then go and inspect the handles to see if sharing them has introduced a security issue.

Modifying a Mapped Section

If you find an interesting `Section` object to modify, you can map it into memory using `Add-NtSection`. But how do you modify the mapped memory? The simplest approach from the command line is to use the `Write-NtVirtualMemory` command, which supports passing a mapped section and an array of bytes to write. **Listing 2-39** demonstrates this technique by assuming you have a handle of interest in the `$handle` variable.

```
PS> $sect = $handle.GetObject()
PS> $map = Add-NtSection -Section $sect -Protection ReadWrite
PS> $random = Get-RandomByte -Size $map.Length
PS> Write-NtVirtualMemory -Mapping $map -Data $random
4096

PS> Out-HexDump -Buffer $map -Length 16 -ShowAddress -ShowHeader
                00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
-----
0000018111C860000: DF 24 04 E1 AB 2A E1 76 EB 19 00 8D 79 28 9C BA
```

Listing 2-39: Mapping and modifying a Section object

We first call the `GetObject` method on the handle to duplicate it into the current process and return a `Section` object. For this to succeed, the process in which we're running this command must be able to access the process with the handle. We then map the handle as `ReadWrite` into the current process's memory.

We can now create a random array of bytes up to the size of the mapped section and write them to the memory region using `Write-NtVirtualMemory`. This is a quick and dirty fuzzer for the shared memory. The hope is that by modifying the memory, the privileged process will mishandle the contents of the memory region. If the privileged process crashes, we should investigate to determine whether we can control the crash using a more targeted modification of the shared memory.

We can display the memory using `Out-HexDump`. One of the useful features of this command over the built-in `Format-Hex` is that it'll print the address in memory based on the mapped file, whereas `Format-Hex` just prints an offset starting at 0.

You can also create a GUI hex editor with the `Show-NtSection` command, specifying a `Section` object to edit. As the section can be mapped into any process, writing it in the GUI hex editor will also modify all other mappings of that section. Here is the command to display the hex editor:

```
PS> Show-NtSection -Section $sect
```

Figure 2-6 shows an example of the editor generated by running the previous command.

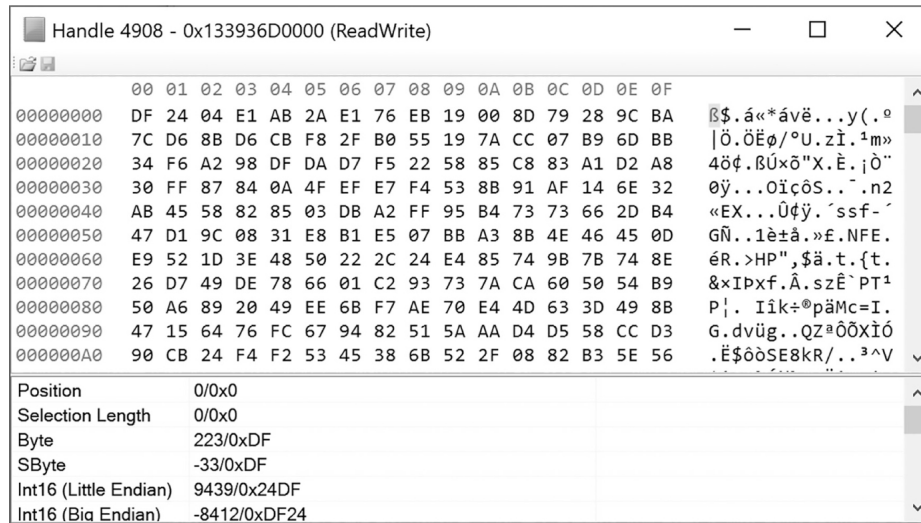


Figure 2-6: The section editor GUI

The GUI shown in **Figure 2-6** maps the section into memory and then displays it in a hex editor form. If the section is writable, you can modify the contents of the memory through the editor.

Finding Writable and Executable Memory

In Windows, for a process to execute instructions, the memory must be marked as executable. However, it's also possible to map the memory as both writable and executable. Malware sometimes uses this combination of permissions to inject shell code into a process and run malicious code using the host process's identity.

Listing 2-40 shows how to check for memory in a process that is both writable and executable. Finding such memory might indicate that something malicious is going on, although in most cases this memory will be benign. For example, the .NET runtime creates writable and executable memory to perform just-in-time (JIT) compilation of the .NET byte code into native instructions.

```

PS> $proc = Get-NtProcess -ProcessId $pid -Access QueryLimitedInformation
PS> Get-NtVirtualMemory -Process $proc | Where-Object {
    $_.Protect -band "ExecuteReadWrite"
}
Address          Size  Protect          Type    State  Name
-----
0000018176450000 4096  ExecuteReadWrite Private Commit
0000018176490000 8192  ExecuteReadWrite Private Commit
0000018176F60000 61440 ExecuteReadWrite Private Commit
--snip--

PS> $proc.Close()

```

Listing 2-40: Finding executable and writable memory in a process

We start by opening a process for `QueryLimitedInformation` access, which is all we need to enumerate the virtual memory regions. Here, we're opening the current PowerShell process; as PowerShell is .NET, we know it will have some writable and executable memory regions, but the process you open can be anything you want to check.

We then enumerate all the memory regions using `Get-NtVirtualMemory` and filter on the `ExecuteReadWrite` protection type. We need to use a bitwise AND operation as there are additional flags that can be added to the protection, such as `Guard`, which creates a guard page that prevents doing a direct equality check.

Wrapping Up

This chapter provided a tour through the Windows kernel and its internals. The kernel consists of many separate subsystems, such as the Security Reference Monitor, the object manager, the configuration manager (or registry), the I/O manager, and the process and thread manager.

You learned about how the object manager manages kernel resources and types, how to access kernel resources through system calls, and how handles are allocated with specific access rights. You also accessed object manager resources through the *NtObject* drive provider as well as through individual commands.

I then discussed the basics of process and thread creation and demonstrated the use of commands such as `Get-NtProcess` to query for process information on the system. I explained how to inspect the virtual memory of a process, as well as some of the individual memory types.

A user doesn't directly interact with the kernel; instead, user-mode applications power the user experience. In the next chapter, we'll discuss the user-mode components in more detail.